Binary File Handling

Binary file operations

- If we want to write a structure such as list or dictionary to a file and read it subsequently we need to use the Python module **pickle**. **Pickling** is the process of converting structure to a byte stream before writing to a file and while reading the content of file a reverse process called **Unpickling** is used to convert the byte stream back to the original format.

- **In Binary files, there is no delimiter for a new line.**

- **Binary files are faster and easier to read and write than a text file.**

Steps to perform binary fileoperations

- First we need to import the modulecalled pickle.
- This module provides 2 main functions:
  - dump() : to write the object in file which is loaded in binary mode
    - Syntax : dump(object_to_write, filehandle)
  - load() : dumped data can be read from file using load() i.e. it is used to read object from picklefile.
    - Syntax: object = load(filehandle)

Example: dump()

```
# program to demonstrate binary file operations
import pickle
myfile = open("project.txt","wb")
dict1 = {'ename':'jitendra','pname':'simulator','charge':45000}
pickle.dump(dict1,myfile)
myfile.close()
```



See the content is some kind of encrypted format, and itis not in complete readable form

Example: load()

```
# program to demonstrate binary file operations
import pickle
mf = open("project.txt","wb")
#dict1 = pickle.load(mf)
#myfile.close()
dict1 = {1:"a",2:"b",3:"c"}
pickle.dump(dict1,mf)
mf.close()
```

```
# program to demonstrate binary file operations
import pickle
mf = open("project.txt","rb")
dict1 = pickle.load(mf)
print(dict1)
print("Item 1 is ",dict1[1])
mf.close()
```

```
{1: 'a', 2: 'b', 3: 'c'}
Item 1 is  a
```

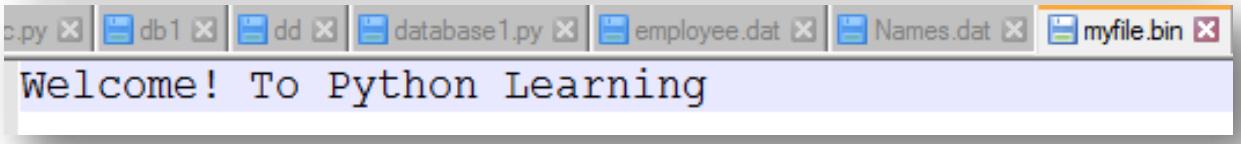
Binary File Handling using encode() and decode() function

Writing String to Binaryfile

- To store string in binary file, we must convert it to binary format either by prefixing the string with ‘b’ or using the encode() function.
- For e.g.

```
# Storing String in Binary File
msg="Welcome!"
f = open('myfile.bin','wb')
f.write(msg.encode())
f.write(b' To Python Learning')
f.close()
```

We can use ‘a’ in place of ‘w’ for append



Reading Binary file in String

```
# Reading String from Binary File
f=open('myfile.bin','rb')
msg=f.read()
print(msg)
print(msg.decode())
f.close()
```

```
b'Welcome! To Python Learning'
Welcome! To Python Learning
```

We can observe, without decoding it will prefix text with ‘b’

Binary File handling using pickle Module

Pickling – Storing employee details in binary file

```
import pickle
emp=[]
f = open('employee.dat','wb')
ans='y'
while ans=='y':
          eno = int(input("Enter Employee Number :"))
          name= input("Enter Employee Name ")
          salary=int(input("Enter Salary "))
          emp.append([eno,name,salary])
          ans=input("Add More record ")
pickle.dump(emp,f)
f.close()
```

```
Enter Employee Number :1
Enter Employee Name AMIT
Enter Salary 8000
Add More record y
Enter Employee Number :2
Enter Employee Name SUMIT
Enter Salary 9000
Add More record y
Enter Employee Number :3
Enter Employee Name NITIN
Enter Salary 45000
Add More record y
Enter Employee Number :4
Enter Employee Name DINKAR
Enter Salary 5500
Add More record n
```

Un-Pickling – Reading and Display Record

```
import pickle
emp=[]
f = open('employee.dat','rb')
ans='y'
while True:
        try:
                emp = pickle.load(f) # loading data in emp list
        except EOFError:
                break
for e in emp:
        print(e)
f.close()
```

```
[1, 'AMIT', 8000]
[2, 'SUMIT', 9000]
[3, 'NITIN', 45000]
[4, 'DINKAR', 5500]
```

Updating Employee Record

```
import pickle
emp=[]
f = open('employee.dat','rb')
emp = pickle.load(f) # loading data in emp list

print("## EMPLOYEE RECORDS ##")
print(emp)
print('-----------------------------------------------------------')
f.close()
f = open('employee.dat','wb')
found=False
en = int(input("Enter Employee Number to update :"))
for i in range(len(emp)):
          if emp[i][0]==en:
                    sl = int(input("Enter New Salary :"))
                    emp[i][2]=sl
                    found=True
                    print("## Record Updated ##")
if not found:
          print("## No Such Employee Number ##")
pickle.dump(emp,f)
f.close()
f=open('employee.dat','rb')
emp=pickle.load(f)
print("## EMPLOYEE RECORDS AFTER UPDATE ##")
print(emp)
print('-----------------------------------------------------------')
```

Updating Employee Record

```
import pickle
emp=[]
f = open('employee.dat','rb')
emp = pickle.load(f) # loading data in emp list

print("## EMPLOYEE RECORDS ##")
print(emp)
print('------------------------------------------------------------')
f.close()
f = open('employee.dat','wb')
found=False
en = int(input("Enter Employee Number to update :"))
for i in range(len(emp)):
            if emp[i][0]==en:
                        sl = int(input("Enter New Salary :"))
                        emp[i][2]=sl
                        found=True
                        print("## Record Updated ##")
if not found:
            print("## No Such Employee Number ##")
pickle.dump(emp,f)
f.close()
f=open('employee.dat','rb')
emp=pickle.load(f)
print("## EMPLOYEE RECORDS AFTER UPDATE ##")
print(emp)
print('------------------------------------------------------------')
```

```
## EMPLOYEE RECORDS ##
[[1, 'amit', 99999], [2, 'nitin', 8000], [3, 'hhh', 5500]]
------------------------------------------------------------
Enter Employee Number to update :2
Enter New Salary :9000
## Record Updated ##
## EMPLOYEE RECORDS AFTER UPDATE ##
[[1, 'amit', 99999], [2, 'nitin', 9000], [3, 'hhh', 5500]]
------------------------------------------------------------
```

Deleting Employee Record

```
import pickle
emp=[]
f = open('employee.dat','rb')
emp = pickle.load(f) # loading data in emp list

print("## EMPLOYEE RECORDS ##")
print(emp)
print('-----------------------------------------------------------')
f.close()
f = open('employee.dat','wb')
found=False
en = int(input("Enter Employee Number to Delete :"))
emp2=[]
for i in range(len(emp)):
          if emp[i][0]!=en:
                    emp2.append(emp[i])

pickle.dump(emp2,f)
f.close()
f=open('employee.dat','rb')
emp=pickle.load(f)
print("## EMPLOYEE RECORDS AFTER DELETE ##")
print(emp)
print('-----------------------------------------------------------')
```

Deleting Employee Record

```
import pickle
emp=[]
f = open('employee.dat','rb')
emp = pickle.load(f) # loading data in emp list

print("## EMPLOYEE RECORDS ##")
print(emp)
print('-----------------------------------------------------------')
f.close()
f = open('employee.dat','wb')
found=False
en = int(input("Enter Employee Number to Delete :"))
emp2=[]
for i in range(len(emp)):
            if emp[i][0]!=en:
                        emp2.append(emp[i])

pickle.dump(emp2,f)
f.close()
f=open('employee.dat','rb')
emp=pickle.load(f)
print("## EMPLOYEE RECORDS AFTER DELETE ##")
print(emp)
print('-----------------------------------------------------------')
```

```
## EMPLOYEE RECORDS ##
[[1, 'AMAN', 5000], [2, 'BIPIN', 9000], [3, 'CHANDU', 7800], [4, 'DINKAR', 9900]]
-----------------------------------------------------------
Enter Employee Number to Delete :3
## EMPLOYEE RECORDS AFTER DELETE ##
[[1, 'AMAN', 5000], [2, 'BIPIN', 9000], [4, 'DINKAR', 9900]]
-----------------------------------------------------------
```